

AI-Orchestrator

Gateway corporativo multi-agente com LLM **100% on-premise**. Uma pergunta em linguagem natural é roteada, executada em paralelo por agentes especialistas sobre microserviços de domínio e sintetizada em uma resposta única — sem nenhum token saindo do perímetro da empresa.

ORQUESTRAÇÃO

LangGraph · Qdrant (rota semântica)

INFERÊNCIA

Ollama · Qwen (local, GPU)

DOMÍNIOS

Finanças · RH · Estoque · Vendas

CUSTO POR CONSULTA

R\$ 0 em API

O PROBLEMA

Perguntas de negócio cruzam sistemas. Pessoas não deveriam precisar cruzá-los manualmente.

Dados fragmentados

"Posso aceitar um pedido de 500 unidades com 15% de desconto?" exige consultar vendas (política), estoque (saldo) e finanças (caixa) — três sistemas, três telas, três equipes.

LLM em nuvem = dado fora

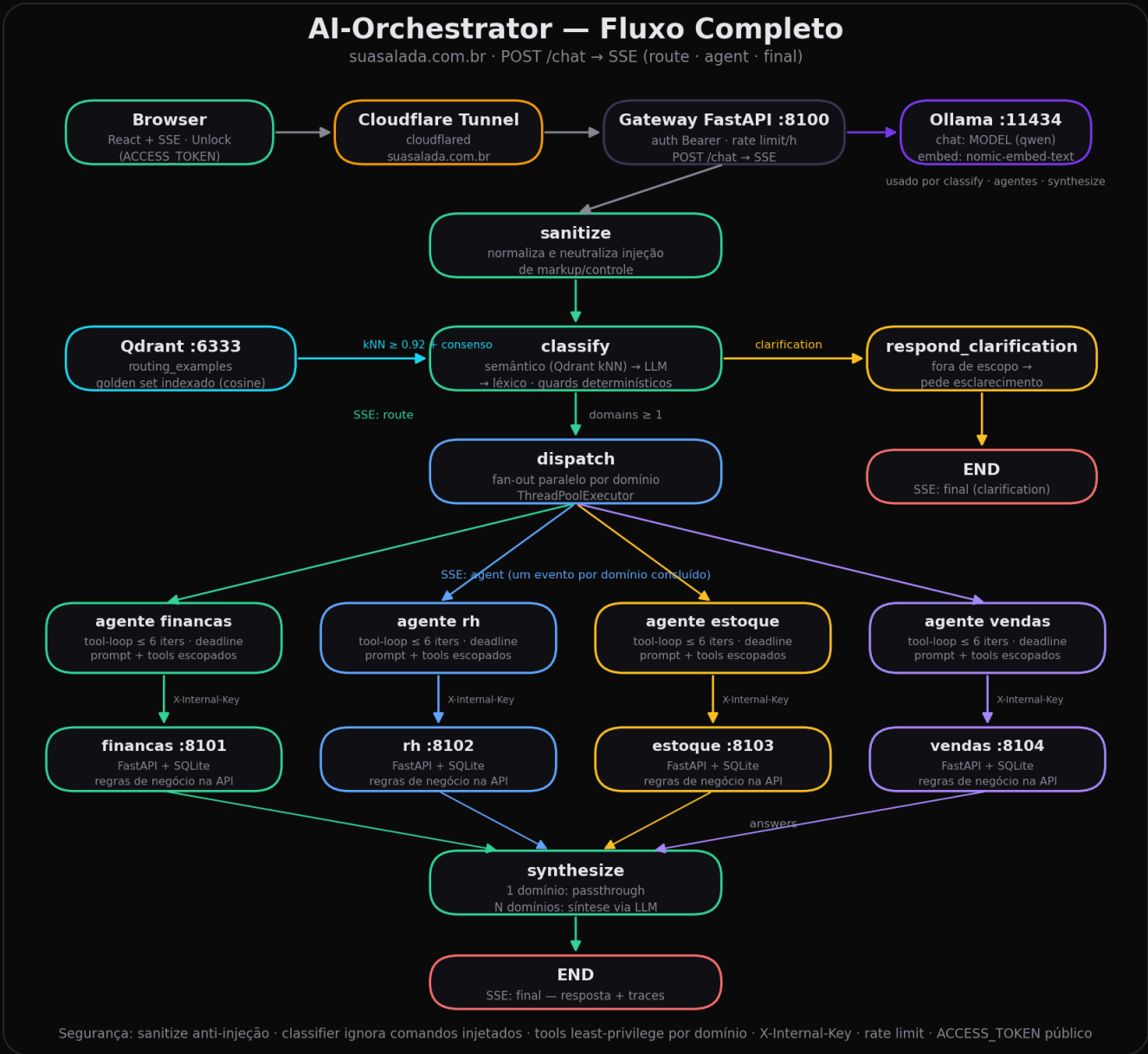
Conectar ChatGPT/Claude direto aos ERPs envia salários, margens e fluxo de caixa para um terceiro. Risco LGPD e custo por token que cresce com o uso.

Regra de negócio no prompt

Abordagens ingênuas codificam alçadas e limites no prompt — o modelo "decide" regra de negócio e alucina valores. Auditoria impossível.

Fluxo completo — da pergunta à resposta auditável

Browser → Cloudflare Tunnel → Gateway FastAPI → grafo LangGraph → agentes em paralelo → microserviços com as regras → síntese. Progresso transmitido por SSE em tempo real (route · agent · final).

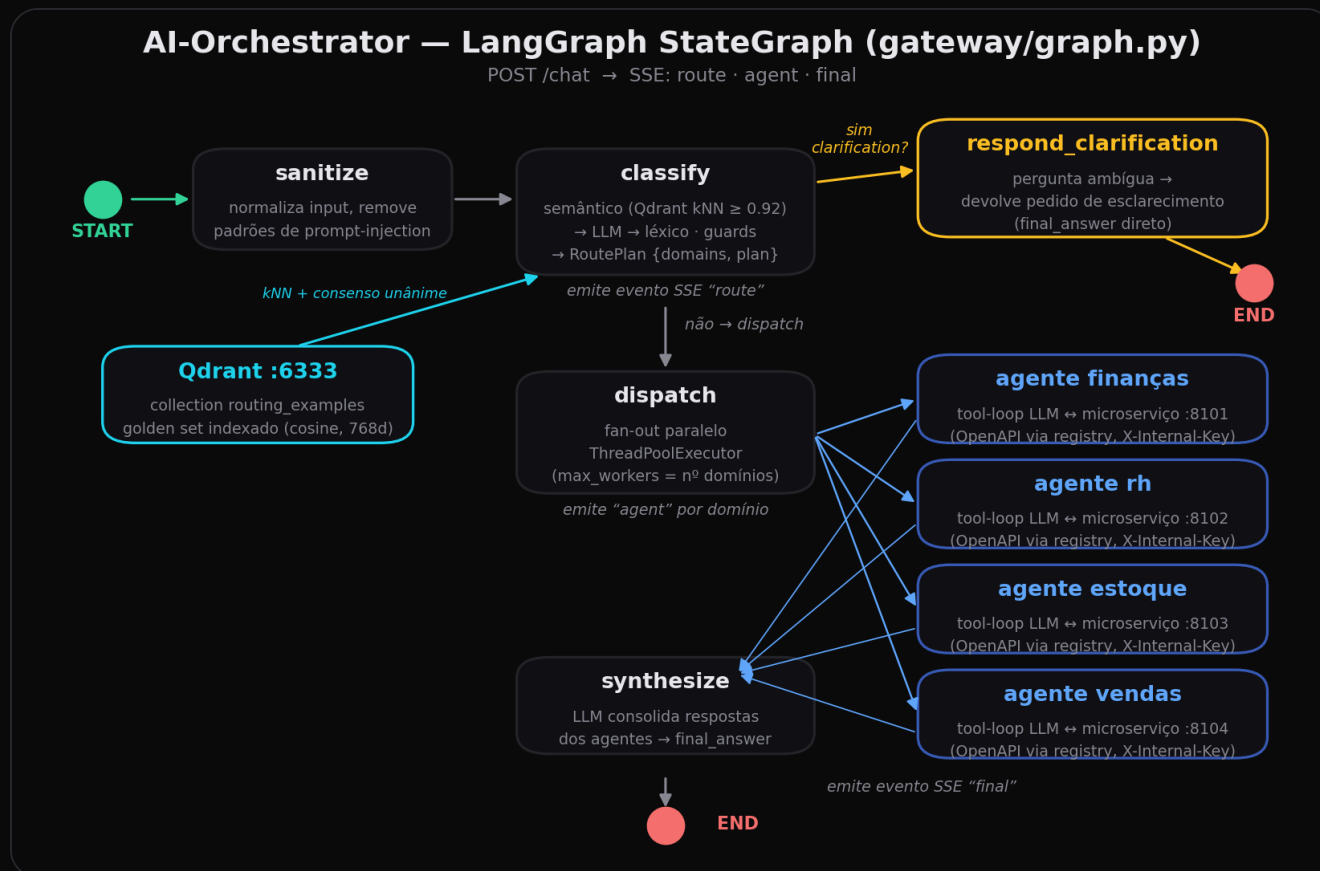


Arquitetura end-to-end em produção. Inferência local via Ollama — nenhum dado sai do perímetro.

ORQUESTRAÇÃO

LangGraph: 5 nós, decisões explícitas

Sanitização anti-injeção → classificação em três camadas (semântica via Qdrant kNN, LLM com retry, fallback léxico determinístico) → fan-out paralelo (ThreadPoolExecutor) → síntese. Pergunta já conhecida resolve em ~0,1 s por similaridade vetorial, sem chamar o LLM. Fora de escopo? O grafo pede esclarecimento em vez de alucinar.

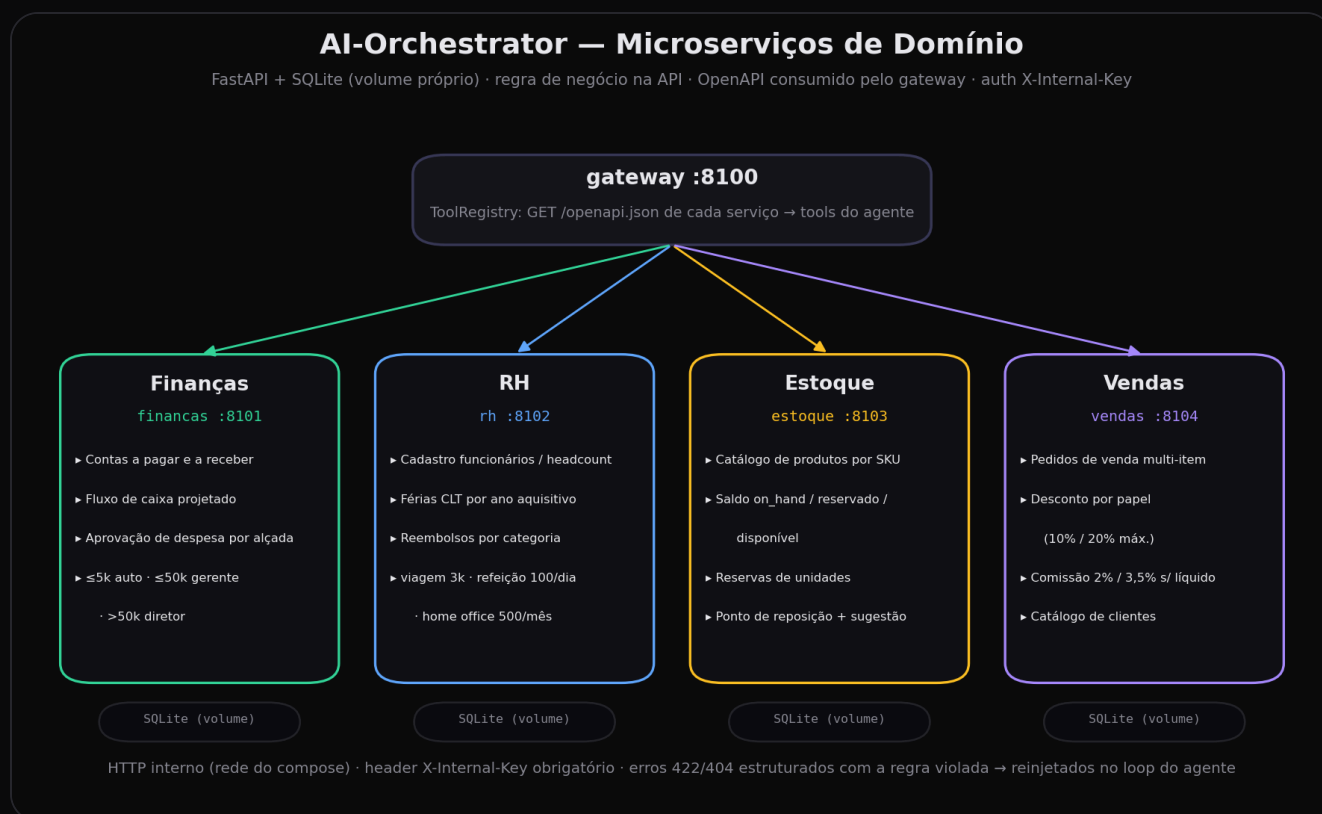


StateGraph do gateway (gateway/graph.py) — cada transição é observável e testada.

DOMÍNIOS

A regra de negócio vive na API — nunca no prompt

Quatro microserviços FastAPI independentes, cada um com seu banco e suas regras (alçadas, CLT, reposição, descontos). O LLM consulta e executa; quem valida é o serviço. Erro 422 volta estruturado com a regra violada e reentra no loop do agente.



Gateway descobre as capacidades de cada serviço via OpenAPI — adicionar um domínio novo não muda o orquestrador.

AGENTES ESPECIALISTAS

Um agente por domínio, least-privilege por construção

Mesmo modelo residente, mas cada agente só enxerga as tools do seu microserviço. Loop limitado (≤ 6 iterações + deadline) e toda afirmação numérica obrigatoriamente rastreável a uma chamada de tool.

Agente Finanças

serviço :8101

Contas a pagar/receber · fluxo de caixa projetado · despesas · alçada de aprovação

FERRAMENTAS (OpenAPI → tools)

- ▶ list_accounts / create_account — contas a pagar e receber
- ▶ settle_account — pay (pagar) / receive (receber)
- ▶ cashflow — projeção de caixa por período

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ $\leq$ R\$ 5.000 → auto-aprovada
- ▶ $\leq$ R\$ 50.000 → exige aprovador 'gerente'
- ▶ $>$ R\$ 50.000 → exige aprovador 'diretor'
- ▶ pay só em conta 'pagar' aberta; receive só em 'receber'

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente RH

serviço :8102

Funcionários · férias CLT · reembolsos · headcount

FERRAMENTAS (OpenAPI → tools)

- ▶ list_employees / get_employee — cadastro e salários
- ▶ vacation_balance / request_vacation — férias por ano aquisitivo
- ▶ create_reimbursement — viagem, refeição, home office

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ 30 dias de férias após 12 meses de admissão
- ▶ fracionamento limitado a N períodos por ano aquisitivo
- ▶ reembolso: $\text{viagem} \leq R3.000 \cdot \text{refeição} \leq R 100/\text{dia}$
- ▶ home office $\leq R\$ 500/\text{mês}$

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente Estoque

serviço :8103

Produtos por SKU · saldo disponível · reservas · ponto de reposição

FERRAMENTAS (OpenAPI → tools)

- ▶ list_products / get_product — catálogo e saldo por SKU
- ▶ create_reservation / release — reserva de unidades
- ▶ replenishment — sugestão de compra

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ available = on_hand – reserved
- ▶ reserva nunca excede o disponível
- ▶ reposição quando available $\leq$ reorder_point
- ▶ sugestão de compra = reorder_point $\times$ 2 – available

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente Vendas

serviço :8104

Pedidos de venda · política de desconto por papel · comissão

FERRAMENTAS (OpenAPI → tools)

- ▶ create_order — pedido com itens, papel e desconto
- ▶ list_orders / get_order — consulta de pedidos

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ desconto $\leq$ 10% 'vendedor' · $\leq$ 20% 'gerente'
- ▶ nunca acima de 20% (hard limit)
- ▶ comissão: 2% até R\$ 10.000 líquido · 3,5% acima

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

SEGURANÇA

Defesa em camadas, validada por evals

Boundary público

- ▶ ACCESS_TOKEN no /chat
- ▶ Rate limit por hora
- ▶ Cloudflare Tunnel — zero porta aberta
- ▶ Serviços internos só em loopback

Anti prompt-injection

- ▶ Nó sanitize neutraliza markup/controle
- ▶ Classificador treinado a ignorar comandos injetados
- ▶ Eval de injeção com golden set versionado

Perímetro interno

- ▶ X-Internal-Key em todo serviço
- ▶ Tools least-privilege por domínio
- ▶ Circuit breaker e timeouts
- ▶ Trace completo de tool-calls por resposta

FERRAMENTAS

Stack 100% open-source — zero licença, zero vendor lock-in

Python · FastAPI

LangGraph

Ollama

Qwen 3.5-9B LoRA (fine-tuned)

Qdrant (busca vetorial)

nomic-embed-text

SQLite

React + Vite

SSE

Docker Compose

Cloudflare Tunnel

pytest + evals (golden sets)

Qualidade contínua

Suíte de testes por camada + evals versionados de roteamento, domínios e injeção — regressão de comportamento do LLM é detectada como bug, não como surpresa.

Modelo configurável + LoRA

Variável MODEL troca o modelo sem código. Produção: **Qwen 3.5-9B LoRA** (fine-tuned em 3.050 exemplos, 100% GPU, ~2–4 s/task). Mesmo grafo, mesmo contrato.

Extensível por OpenAPI

Novo domínio = novo microserviço com OpenAPI. O ToolRegistry gera as tools automaticamente; o orquestrador não muda.

On-premise: custo marginal por consulta → zero

Cada consulta multi-domínio dispara 8–15 chamadas de LLM (classificação, loops de tools, síntese) — em API de nuvem, isso é caro e cresce linearmente com o uso.

CENÁRIO (1.000 CONSULTAS/MÊS)	CUSTO MENSAL ESTIMADO*	DADOS SAEM DO PERÍMETRO?
API frontier (GPT-4o / Claude Sonnet) ~40k tokens por consulta multi-domínio	R\$ 800 – 1.600	Sim
GPU em nuvem 24/7 (A10G, g5.xlarge ~US\$ 1/h)	~R\$ 4.000	Não (mas infra de terceiro)
Esta arquitetura — GPU própria (RTX 3060 12 GB): energia + amortização	~R\$ 100 (50 energia + 50 amortização)	Não

R\$ 0

de API e de licença de software. Stack inteira open-source; Cloudflare Tunnel no plano gratuito; domínio R\$ 40/ano.

~R\$ 100/mês

TCO mensal amortizado: energia + hardware (~R\$ 1.800 ÷ 36 meses). 8–16× mais barato que API frontier; ~40× que GPU em nuvem.

LGPD ✓

salários, margens e caixa nunca deixam a rede. Sem DPA com terceiros, sem risco de retenção de prompt, custo de compliance reduzido.

* Estimativas com câmbio R\$ 5,30/US\$. API: 40k tokens/consulta (medido na demo com fan-out de 3 domínios) a preços públicos de GPT-4o (US\$ 2,50/M input, US\$ 10/M output). Energia: ~170 W de GPU em uso intermitente, tarifa média residencial. Amortização: RTX 3060 (~R\$ 1.800) linear em 36 meses de vida útil, valor residual zero. O custo de API escala linearmente com o volume; o on-premise é praticamente flat até a saturação da GPU — quanto maior o uso, maior a economia.

CONCLUSÃO

IA corporativa com soberania de dados, custo previsível e regra de negócio auditável.

Tese arquitetural

LLM como orquestrador de capacidades, não como dono da verdade: o modelo conversa, as APIs decidem. Isso elimina alucinação de regra e torna cada resposta rastreável.

Tese econômica

Para cargas internas e dados sensíveis, inferência local em hardware commodity vence nuvem em custo total já no primeiro mês — e a economia cresce com o uso.

Em produção

Live em **suasalada.com.br** — gateway, 4 domínios, streaming SSE e tunnel, rodando em uma única máquina com RTX 3060.